École d'ingénieur Denis Diderot
Université Paris Cité

Projet Méthodes Numériques
2024-2025

Anneaux de Saturne

Yagarni JEYASEELAN & Meyssem SOUSSOU

SOMMAIRE

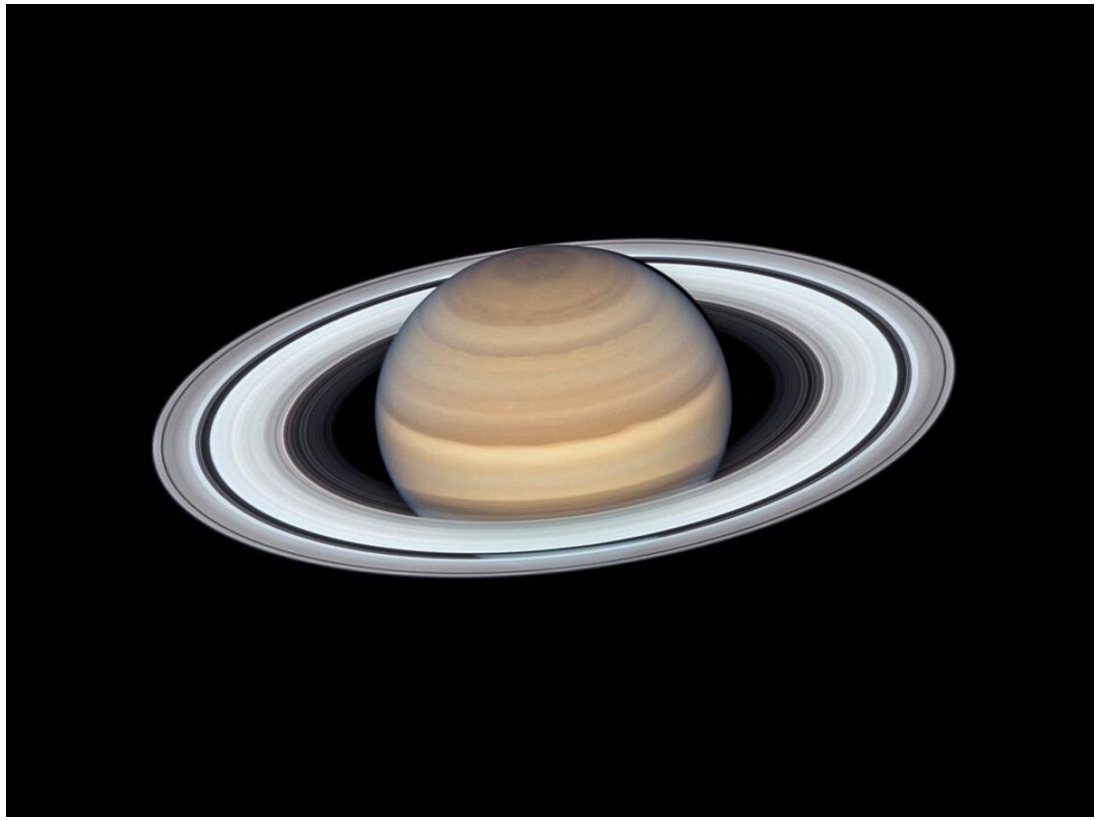
- Résumé
- Introduction
- Modélisation physique
- Développement informatique
- Application de la troisième loi de Kepler
- Visualisation et traitement des résultats
- Résultats et observations
- Discussion et perspectives
- Conclusion

RÉSUMÉ

- **Objectif** : simuler le mouvement d'astéroïdes autour de Saturne perturbé par un satellite.
- **Méthodes** : gravitation classique + intégration Runge-Kutta 4.
- Vitesses initiales via la 3e loi de Kepler.
- Visualisation avec Python.

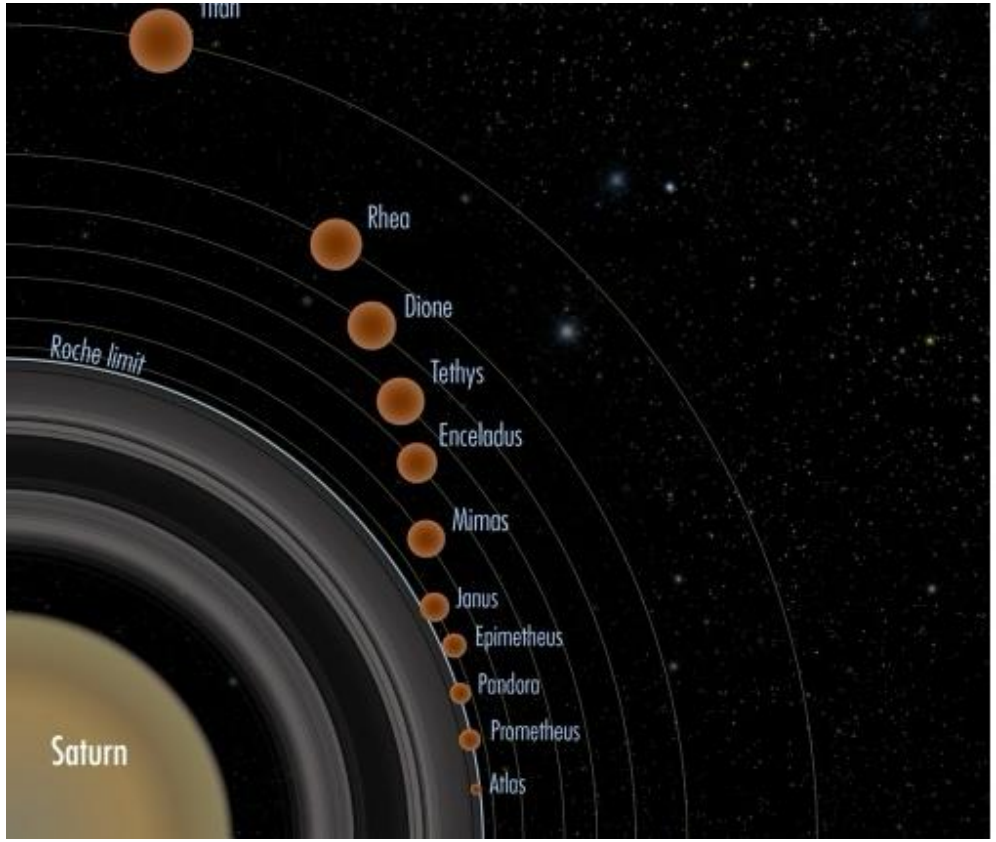
INTRODUCTION

- **Anneaux** : milliards de particules influencées par la gravitation
- **Étude** : effet d'un satellite sur des orbites circulaires
- Méthodes numériques pour simuler le comportement



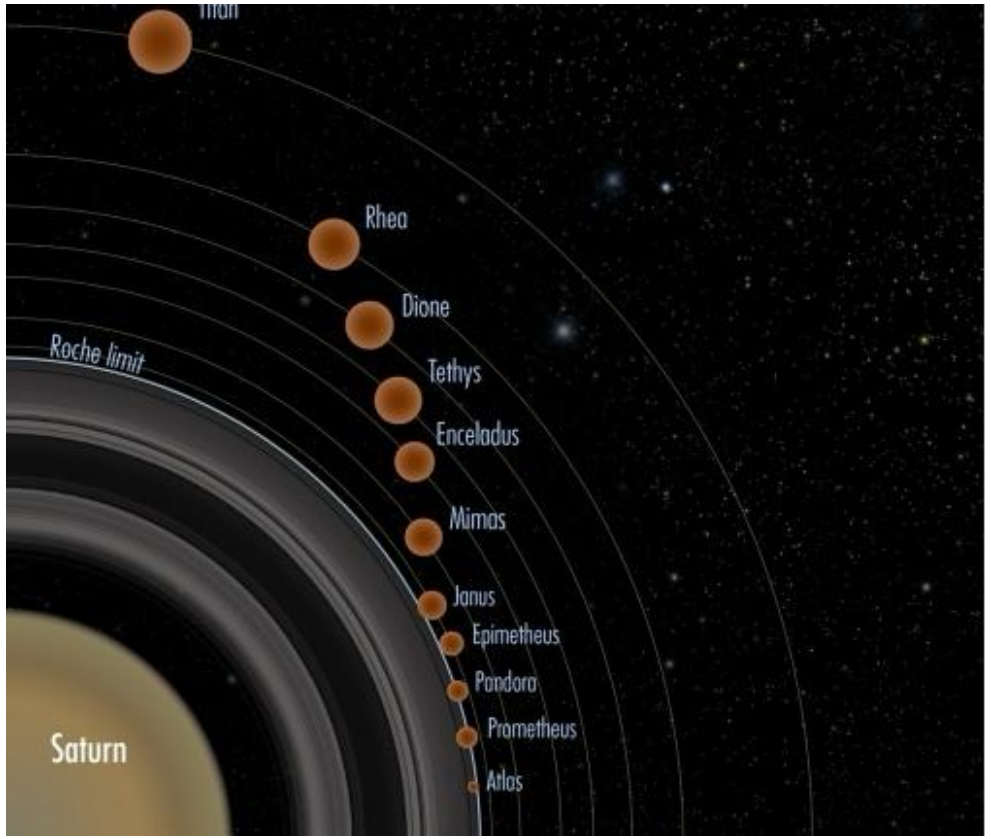
MODÉLISATION PHYSIQUE

- Saturne fixe au centre
- Satellite en orbite
- 10 astéroïdes sur orbites circulaires
- **Forces appliquées** :
  - Gravitation de Saturne
  - Gravitation du satellite



HYPOTHÈSE SIMPLIFICATRICES

- Pas d'interaction entre astéroïdes
- Pas de collisions
- Saturne immobile
- Mouvement sur le plan (x,y)



DÉVELOPPEMENT INFORMATIQUE

- **Langage** : C
- Structure vecteur pour positions/vitesses
- Fonction acceleration() : combine les effets gravitationnels

DÉVELOPPEMENT INFORMATIQUE

- Intégration Runge-Kutta 4 (RK4) :
  - Haute précision
  - Stabilité énergétique
- Initialisation :
  - Positions aléatoires
  - Vitesses via Kepler
  - Pas de temps : 10 s

DÉVELOPPEMENT INFORMATIQUE

trajectoire.c

- **But** : Simuler un seul astéroïde autour de Saturne
- **Satellite** : Aucun
- **Méthode numérique** : RK4
- **Pas de temps** : Très large (DT = 8000 s)
- **Remarque** : Simulation de base sans perturbation ni complexité

```c
// Accélération gravitationnelle due à Saturne
vect acceleration(vect pos) {
    double r = sqrt(pos.x*pos.x + pos.y*pos.y);
    double F = -G*MS/(r*r*r);
    vect acc = {F*pos.x, F*pos.y};
    return acc;
}

// Méthode Runge-Kutta 4
void RK4(vect *pos, vect *vit, double dt) {
    vect k1_v = acceleration(*pos);
    vect k1_r = *vit;
    ...
    // idem pour k2, k3, k4
}
```

DÉVELOPPEMENT INFORMATIQUE

trajectoire_avec satellite.c

- **But** : Simuler un astéroïde perturbé par un satellite
- **Satellite** : Oui (masse = 1.35e23 kg)
- **Méthode numérique** : Runge-Kutta 4 (RK4)
- **Particularité** : Le satellite n'a pas d'effet sur lui-même
- **Pas de temps** : DT = 8000 s
- **Remarque** : Introduction conditionnelle de la gravité du satellite via *isAsteroid*

```c
// Accélération avec influence conditionnelle du satellite
vect acceleration(vect pos, vect sat_pos, int isAsteroid) {
    ...
    if (isAsteroid) {
        double dx = sat_pos.x - pos.x;
        ...
        acc.x += F_sat*dx;
        acc.y += F_sat*dy;
    }
    return acc;
}

// Utilisation dans RK4
RK4(&ast_pos, &ast_vit, sat_pos, DT, 1); // astéroïde
RK4(&sat_pos, &sat_vit, sat_pos, DT, 0); // satellite
```

DÉVELOPPEMENT INFORMATIQUE

trajectoire_avec_satellite_euler.c

- **But** : Même simulation que le précédent mais avec **Euler**
- **Méthode numérique** : **Euler explicite** (moins précis)
- **Satellite** : Oui (masse = 2.3e20 kg)
- **Pas de temps** : DT = 10 s, mais N = 1 000 000 itérations
- **Remarque** : Sert à comparer la précision RK4 vs Euler

```
// Méthode d'Euler
void euler(vect *pos, vect *vit, vect sat_pos, double dt, int isAsteroid) {
    vect acc = acceleration(*pos, sat_pos, isAsteroid);
    vit->x += acc.x*dt;
    pos->x += vit->x*dt;
}
```

DÉVELOPPEMENT INFORMATIQUE

aste2.c

- **But** : Simuler 10 astéroïdes autour de Saturne **sans** satellite
- **Méthode numérique** : RK4
- **Satellite** : positionné mais masse = 0 (donc ignoré)
- **Initialisation** : Positions/vitesses aléatoires dans une plage large
- **Remarque** : Version multiasteroid sans perturbation

```c
#define NUM_ASTEROIDES 10
...
double speed = sqrt(G*MS/distance);
ast_pos[i].x = distance*cos(angle);
ast_vel[i].x = -speed*sin(angle);

// Simulation
for (int j = 0; j < NUM_ASTEROIDES; j++) {
    rk4(&ast_pos[j], &ast_vel[j], sat_pos, DT);
}
```

DÉVELOPPEMENT INFORMATIQUE

aste3.c

- **But** : Même chose que aste2.c mais **avec satellite perturbateur**
- **Satellite** : Oui, masse significative (2.3e20 kg)
- **Initialisation** : Orbites bien espacées (entre 120 000 et 210 000 km).
- **Méthode** : RK4
- **Remarque** : Simulation complète et cohérente avec le rapport scientifique

```c
// Accélération combinée Saturne + satellite
Vector3D acceleration(Vector3D pos, Vector3D sat_pos) {
    ...
    acc.x += factor_s * pos.x;     // Saturne
    acc.x += factor_sat * dx;      // Satellite
    return acc;
}

// Simulation avec satellite en mouvement
rk4(&sat_pos, &sat_vel, (Vector3D){0,0}, DT);
```

DÉVELOPPEMENT INFORMATIQUE

astéroides.c

- **But** : Simuler 10 astéroïdes perturbés par un satellite
- **Satellite** : Oui
- **Méthode numérique** : RK4
- **Initialisation** : Aléatoire, distances plus larges
- **Remarque** : Logique identique à trajectoire_avec_satellite.c mais multi-astéroïdes

```c
// Initialisation aléatoire des astéroïdes
for (int i = 0; i < ASTE; i++) {
    double angle = (rand()%360)*M_PI/180.0;
    double vitesse = 9.0 + (rand()%4);
    ast_pos[i].x = distance*cos(angle);
    ast_vit[i].x = -vitesse*cos(angle);
}
```

3e LOI DE KEPLER

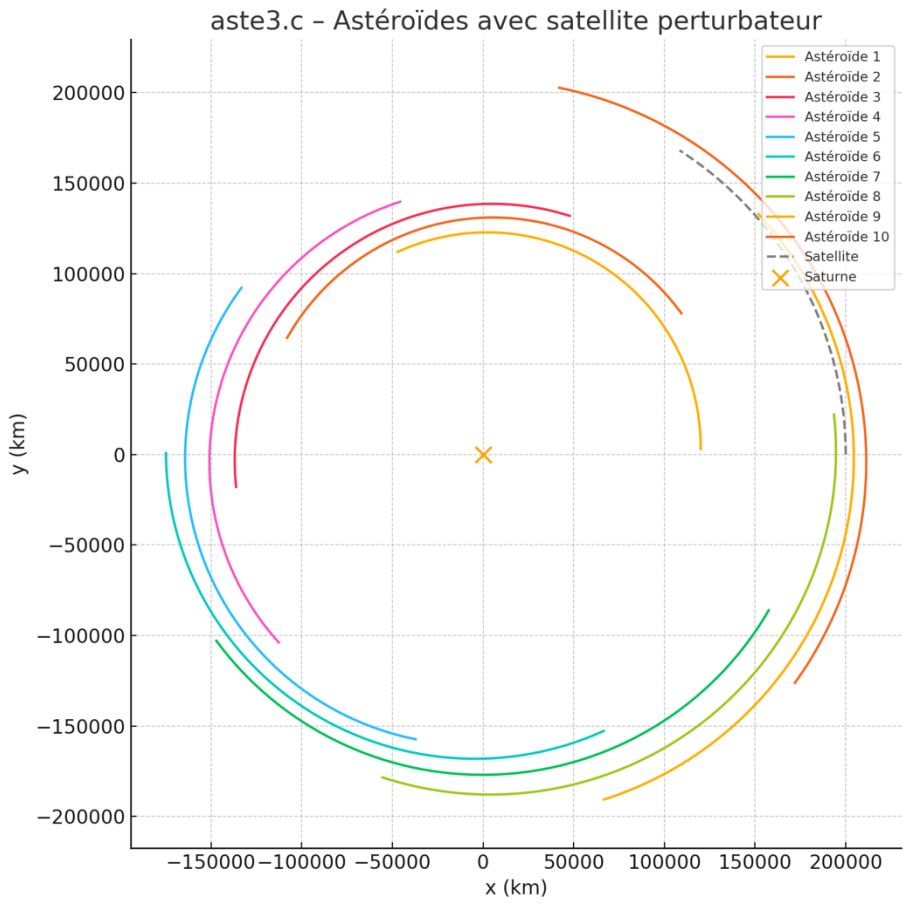
- **Formule** : $v = \sqrt{(GM/r)}$
- Application pour initier les vitesses orbitaires
- Mouvement circulaire garanti

$$\frac{T^2}{a^3} = \frac{4\pi^2}{GM} = \text{constante}$$

$$F(\mathrm{A/B}) = F(\mathrm{B/A}) = \frac{G \cdot m(\mathrm{A}) \cdot m(\mathrm{B})}{d^2}$$

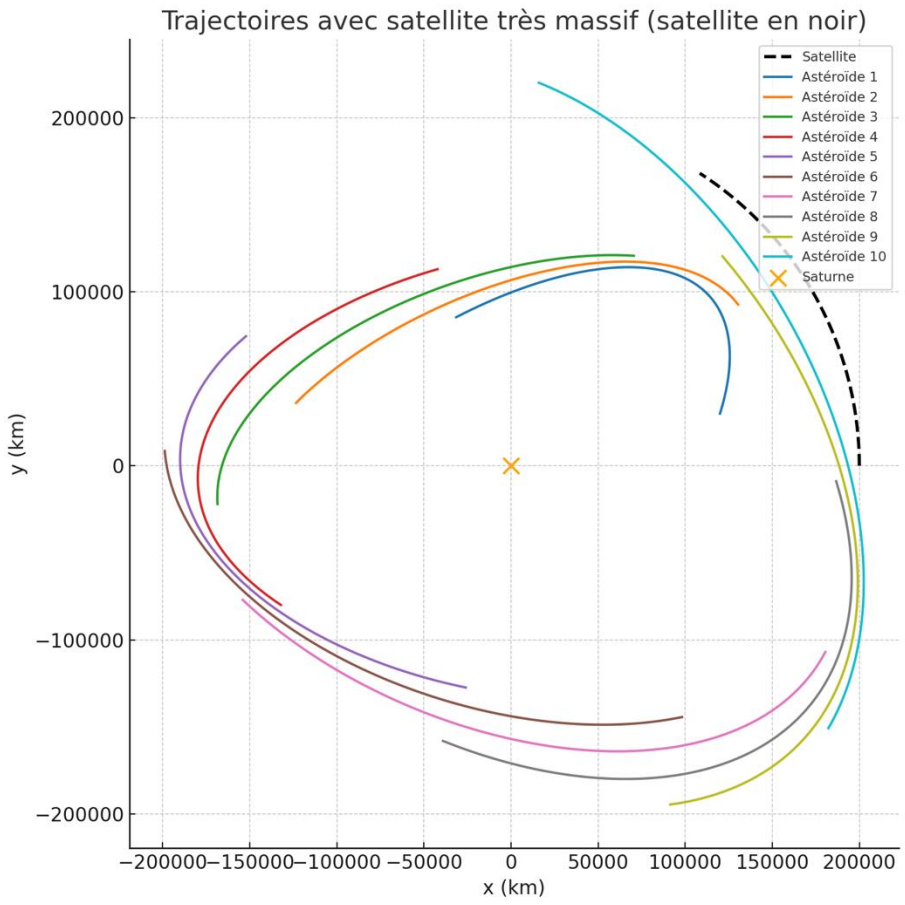
VISUALISATION

- Données sauvegardées à chaque étape
- Utilisation de numpy et matplotlib
- Représentation 2D des trajectoires



RÉSULTATS

- Orbites stables et circulaires
- Perturbations visibles lors du passage du satellite



DISCUSSION ET PERSPECTIVES

- Limites :
  - Pas de collisions, pas de gravitation entre astéroïdes
- Améliorations :
  - Modèle de collisions, forces de répulsion
  - Étude de résonances complexes

CONCLUSION

- Simulation fidèle de la dynamique orbitale
- Méthode **RK4** : précise et stable
- Ouvre la voie à des simulations plus riches